CHAPTER 1

Instruction Tuning and Model Alignment

1.1 The Alignment Problem: From Next-Token Prediction to Assistants

1.1.1 Why autoregressive pre-training yields unaligned models

Autoregressive pre-training optimizes the log-likelihood of sequential text tokens over massive, uncurated web corpora. From a philosophical perspective, this objective forces the model to perform statistical pattern matching across linguistic surfaces rather than acquire genuine semantic comprehension, epistemic awareness, or communicative intent. As formalized in information theory and natural language philosophy, predicting the statistically most probable continuation token is fundamentally distinct from understanding the meaning of a communicative act or recognizing the pragmatic role of an assistant. The network learns the statistical distribution of strings as they appear in the wild—encompassing internet debates, fictional dialogues, academic worksheets, and machine-generated spam—without grounding those strings in external reality or attributing agency to the user. Consequently, when prompted, an autoregressive base model operates strictly under the paradigm of unconditional document completion, remaining completely oblivious to whether the prompt constitutes a query, an imperative command, or an unfinished thought.

From a mathematical perspective, standard pre-training minimizes the negative log-likelihood (NLL) over sequence tokens:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\sum_{t=1}^{N} \log P_\theta(w_t \mid w_{<t}), \tag{1.1}$$

where $P_\theta(w_t \mid w_{<t}) = \text{softmax}\left(W_{\text{unembed}} h_t\right)$ represents the categorical distribution over the vocabulary $\mathcal{V}$.

Equation (1.1) exhibits critical mathematical deficiencies when viewed as an alignment objective:

1. **Equal Penalty for Semantically Divergent Errors**: The cross-entropy loss penalizes every token $w \neq w_t^*$ based purely on its predictive probability mass, treating all non-target tokens with equal mathematical hostility. If the ground-truth token is a valid word, predicting an elegant, factually accurate synonym receives a substantial cross-entropy penalty simply because it does not match the exact surface token of the training document. Conversely, predicting a catastrophic factual lie, a dangerous hallucination, or an aggressive insult receives the exact same magnitude of penalty as a harmless lexical variation. Equation (1.1) possesses no mathematical metric for truthfulness, helpfulness, or qualitative severity.

2. **Imitation of Statistical Modes across Heterogeneous Quality**: Internet text is inherently noisy, containing conflicting viewpoints, logical fallacies, offensive diatribes, and unverified rumors. Because maximum likelihood estimation forces the network to minimize the forward Kullback–Leibler divergence $D_{\text{KL}}\left(P_{\text{data}} \parallel P_\theta\right)$, the model is mathematically

penalized if it fails to place probability mass on all modes of the training distribution. It learns to mimic low-quality, untruthful, and toxic text with the exact same fidelity that it mimics rigorous scientific discourse.

3. **Absence of Intent and Communicative Conditioning**: The loss function contains no term that models the user's implicit goal or communicative intention. The model optimizes sequence continuation conditioned exclusively on the prefix tokens, lacking any loss gradient that rewards task completion, conciseness, or user satisfaction.

This profound structural discrepancy is termed the **intent gap** [**?**]: the divergence between what the human prompter wants the model to accomplish and what the autoregressive loss trains the model to do.

Pedagogical Example: Base Model Prompt Continuation versus Assistant Directive Execution

To observe the practical manifestation of the intent gap, consider submitting an elementary factual query to a base language model (such as base LLaMA-2 7B prior to instruction tuning):

```
"what is 2 + 2?"
```

Because the model was trained exclusively to continue web documents, it does not interpret the string as an imperative request to calculate an arithmetic sum. Instead, observing that short mathematical expressions on the internet frequently occur within repetitive worksheets, exam practice tests, or FAQ forum posts, the unaligned base model continues the text with statistically correlated questions:

```
What is 2 plus 2?  What is the answer to 2 plus 2?  What is the answer to
                              2 plus 2 in math?
```

In contrast, an instruction-aligned assistant interprets the communicative intent behind the prompt, suppresses prompt repetition, and outputs the direct, helpful answer:

```
The answer to 2 + 2 is 4.
```

As illustrated in Figure 1.1, increasing parameter capacity from 7B to 175B does not bridge this gap. An unaligned 175B model generates more fluent, grammatically sophisticated worksheet questions, but still fails to execute the communicative directive.

1.1.2 Empirical demonstration of non-alignment: the format fragility of T5

The limitations of early sequence-to-sequence conditioning were brought to light by the **Text-to-Text Transfer Transformer (T5)** [**?**]. T5 sought to unify all natural language processing tasks under a single text-to-text encoder-decoder framework by prepending a rigid, hardcoded prefix string to every input:

Translation: `"translate English to German: That is good."` ⟶ `"Das ist gut."`

Classification: `"cola sentence: The course is terrible."` ⟶ `"unacceptable"`

Summarization: `"summarize: State authorities announced..."` ⟶ `"Authorities issued a state.`

While T5 proved that diverse tasks could share identical model weights, digging deeply into its behavior reveals that T5 was fundamentally **unaligned and brittle**. T5 did not learn generalized instruction comprehension. Instead, the prefix strings acted merely as fixed associative lookup keys that routed activations through specific parameters. The moment user inputs deviated even slightly from the exact syntax seen during training, T5's execution collapsed catastrophically.

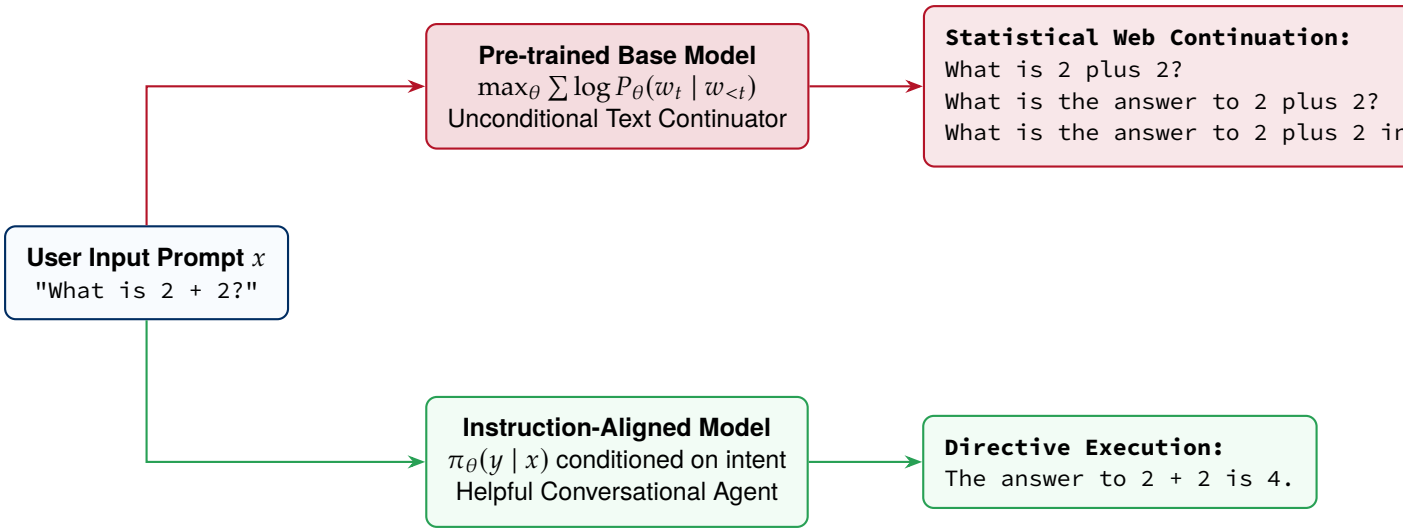


Figure 1.1: The intent gap between autoregressive next-token prediction and instruction alignment. When prompted with an elementary arithmetic query, an unaligned base foundation model treats the prompt as an open-ended document prefix and synthesizes statistically correlated worksheet questions. Conversely, an aligned assistant interprets the prompt as an imperative task directive, returning the exact communicative answer.

Pedagogical Example: T5 Prompt Brittleness and Format Fragility

The empirical fragility of T5's prefix mechanism is evident in three canonical failure modes (documented in course lecture slides):

1. **Canonical Pre-trained Prefix (Exact Syntax)**:

   Input: `"can you translate from English to German, What is your profession?"`
   Output: `"Was ist Ihr Beruf?"` (Correct execution)

2. **Minor Natural Language Paraphrase**:

   Input: `"can you translate from English to German the following sentence? What is y`
   Output: `"<unk> <unk> <unk>..."` (Catastrophic failure into unknown tokens)

3. **Novel Task Directive**:

   Input: `"compute: 2+2 ="`
   Output: `":2+2+2+2+2+2+2+2+2+..."` (Degeneration into an infinite loop)

The failure cases in Box 1.1.2 demonstrate that T5 lacked both **cross-task generalization** and **phrasing robustness**. Adding subordinate clauses (such as `"the following sentence?"`) broke the associative key, causing the network to assign probability mass to out-of-vocabulary tokens. Furthermore, T5 could not generalize to tasks outside its pre-defined prefix dictionary (such as performing arithmetic via `"compute:"`), generating degenerative repetitive patterns instead.

1.1.3 Defining alignment: the HHH framework

To establish a principled engineering foundation for transforming unaligned predictors into effective assistants, Askell et al. [**?**] formulated the **HHH alignment framework**. An aligned model is defined as an agent whose outputs satisfy three complementary criteria:

Definition 1.1: *The HHH Alignment Criteria*

A language model is considered aligned if its behavior simultaneously satisfies:

1. **Helpfulness**: The model makes an active, high-fidelity effort to resolve the user's objective. It adheres strictly to formatting, length, and stylistic constraints, asks for clarification when requests are genuinely underspecified, avoids unsolicited conversational preamble, and maintains appropriate conciseness.

2. **Honesty**: The model produces claims that are factually grounded and accurate. Crucially, it must exhibit calibrated uncertainty, explicitly stating when information is unknown or ambiguous rather than fabricating plausible-sounding hallucinations or false citations.

3. **Harmlessness**: The model refuses to generate content that facilitates physical harm, cyberattacks, weapons synthesis, illegal acts, or severe psychological harassment, while avoiding unjustified refusals on benign, academic, or adjacent safety queries.

In practical alignment engineering, the HHH criteria exist in Pareto tension. An uncalibrated focus on harmlessness leads to **over-refusal pathologies**, where the assistant rejects benign coding queries (such as debugging network socket code or parsing vulnerability signatures). Conversely, naive helpfulness can assist malicious users in dangerous activities. Alignment techniques must navigate this Pareto frontier to balance safety with functional utility.

1.1.4 In-context few-shot learning and its three fundamental limitations

Before the advent of parametric instruction tuning, researchers attempted to steer base models toward task execution using **in-context few-shot learning** [**?**]. By prepending K demonstration exemplars to the prompt:

$$x_{\text{prompt}} = \left(x^{(1)}_{\text{demo}}, y^{(1)}_{\text{demo}}\right) \circ \left(x^{(2)}_{\text{demo}}, y^{(2)}_{\text{demo}}\right) \circ \cdots \circ \left(x^{(K)}_{\text{demo}}, y^{(K)}_{\text{demo}}\right) \circ x_{\text{query}}, \tag{1.2}$$

the internal self-attention heads track the demonstration pattern, guiding the model to generate completion y_{query} without modifying network weights θ.

While few-shot prompting demonstrated that foundation models possess latent multi-task capabilities, relying on in-context learning to build conversational assistants suffers from **three fundamental limitations**:

1. **Severe Instability and Format Brittleness**: In-context few-shot predictions are notoriously unstable. Empirical evaluations show that accuracy fluctuates drastically depending on the arbitrary ordering of exemplars, minor variations in whitespace or delimiter tokens, and the frequency of class labels within the prompt. Few-shot models exhibit severe **recency bias** (disproportionately predicting the label of the final exemplar) and **majority-class bias**, making production deployments unreliable.

2. **Complete Failure for Instructions Lacking Natural Exemplars**: Many real-world human interactions intrinsically do not admit few-shot demonstrations. Prompts involving open-ended creative synthesis (e.g., `"Draft a persuasive diplomatic communique`

`regarding deep-sea mining rights`"), dynamic full-codebase refactoring, or bespoke multi-turn conversational troubleshooting cannot be preceded by realistic exemplars without either pre-solving the unique task or consuming the entire context window. Expecting users to write $K = 5$ expert demonstrations before asking a question defeats the purpose of an automated assistant.

3. **Complete Failure to Generalize to Unseen Tasks in Zero-Shot Regimes**: In the true zero-shot setting ($K = 0$)—where the model receives an instruction without demonstration pairs—base models fail completely. Because they were never trained to associate imperative natural language with task execution, base models revert to document continuation, echoing the prompt or drifting into irrelevant topics.

These three bottlenecks established the urgent necessity of **parametric instruction tuning**: permanently modifying the neural network weights θ so that the model comprehends and executes instructions in zero-shot mode.

1.2 Instruction Tuning: T0, FLAN, and Cross-Task Generalization

1.2.1 The mechanics of instruction tuning: multi-templates and task inversion

Instruction tuning reframes supervised learning by converting traditional, tabular NLP datasets (classification, extraction, inference) into natural language prompt-response pairs. Instead of training separate heads on numeric labels, every task is expressed as a conversational command.

From a philosophical perspective, instruction tuning forces the neural network to move beyond surface-level lexical memorization. In natural human language, an underlying communicative intent can be expressed through infinitely many syntactic permutations. By training a model on multiple, diverse phrasings of the exact same underlying task, the network is compelled to abstract the **semantic essence** of the instruction, disentangling the operation to be performed from the surface idiosyncrasies of word choice.

From a mathematical perspective, multi-template instruction tuning serves as a powerful form of **semantic data augmentation and gradient regularization**. Let $\mathcal{D}_k$ denote a supervised task dataset, and let $\{\mathcal{T}_{k,1}, \mathcal{T}_{k,2}, \ldots, \mathcal{T}_{k,M_k}\}$ represent a collection of M_k distinct natural language templates that map an input-output pair $(x, y) \in \mathcal{D}_k$ into a prompt sequence $\tilde{x} = \mathcal{T}_{k,m}(x)$ and target completion y. The multitask instruction tuning objective is:

$$\mathcal{L}_{\text{IT}}(\theta) = -\sum_{k=1}^{K} \frac{1}{M_k} \sum_{m=1}^{M_k} \mathop{\mathbb{E}}_{(x,y)\sim\mathcal{D}_k} \left[\sum_{t=1}^{|y|} \log P_\theta \left(y_t \,\middle|\, \mathcal{T}_{k,m}(x), y_{<t} \right) \right]. \tag{1.3}$$

When trained on a single template ($M_k = 1$), the gradient update $\nabla_\theta \mathcal{L}$ couples the task logic to specific function words present in the template. The self-attention projection matrices W_Q, W_K learn to attend heavily to fixed trigger tokens. Conversely, when $M_k \geq 10$, the model is exposed to varied grammatical structures (imperative commands, interrogative questions, affirmative assertions). The loss forces the intermediate hidden representations h_t to converge to an invariant task manifold $\mathbf{z}_{\text{task}}$, ensuring that:

$$\|h(\mathcal{T}_{k,m}(x)) - h(\mathcal{T}_{k,j}(x))\|_2 \ll \|h(\mathcal{T}_{k,m}(x)) - h(\mathcal{T}_{l,m}(x))\|_2 \quad \forall m \neq j,\ k \neq l. \tag{1.4}$$

A crucial component of this methodology is **task inversion** (or "task flipping"). In task inversion, the standard input-output relationship is reversed: the model is provided with the target label or answer and instructed to reconstruct a plausible input query:

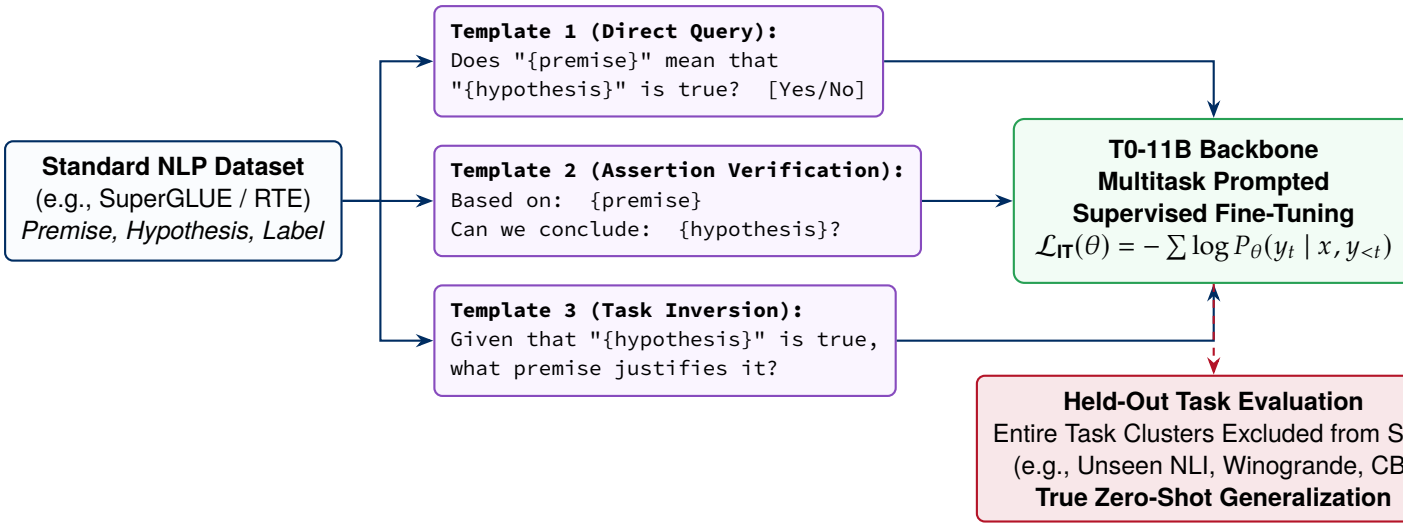


Figure 1.2: The T0 multitask prompted fine-tuning architecture [**?**]. Conventional tabular NLP datasets are mapped into conversational queries through a diverse collection of crowdsourced prompt templates, including direct questions, verification assertions, and task inversions. Fine-tuning on these multi-prompt formulations enables the model to generalize zero-shot to entirely held-out task clusters without seeing task-specific exemplars.

```
"Given the answer '<answer>', what would be a logical question that produced
                                    it?"
```

Task inversion prevents the model from developing unidirectional heuristic shortcuts, enforcing bidirectional semantic consistency across the latent representation space.

1.2.2 T0: multitask prompted training and prompt robustness

Sanh et al. [**?**] implemented this vision in **T0**, utilizing an 11-billion parameter T5 encoder-decoder backbone. T0 investigated two central hypotheses:

1. Can fine-tuning on a massively diverse mixture of prompted tasks enable zero-shot generalization to unobserved, held-out task families?

2. Does training on multiple prompt formulations per dataset confer robustness against arbitrary prompt wording?

To evaluate cross-task generalization without data leakage, Sanh et al. established a strict **held-out task evaluation protocol**. All datasets belonging to an entire capability cluster—specifically Natural Language Inference (NLI), Coreference Resolution, and Word Sense Disambiguation—were completely excluded from the fine-tuning mixture. T0 was fine-tuned exclusively on question answering, summarization, and sentiment classification benchmarks using crowdsourced templates from the *PromptSource* repository ($\approx 8 - 10$ templates per dataset).

When evaluated on the held-out NLI datasets (such as RTE, CB, and ANLI), T0-11B demonstrated massive zero-shot generalization. It outperformed its untuned T5-11B baseline by over 15% absolute accuracy and decisively surpassed the zero-shot performance of the 175-billion parameter GPT-3 model, despite operating with 16× fewer parameters.

Furthermore, T0 established the empirical law of **prompt robustness**. As plotted in Figure 1.3, when models are fine-tuned with only a single prompt template per dataset ($p = 1$), they remain highly sensitive to evaluation phrasing, exhibiting a broad variance envelope ($> 13\%$ spread). Scaling the number of templates to $p = 5.7$ and $p = 8.03$ simultaneously

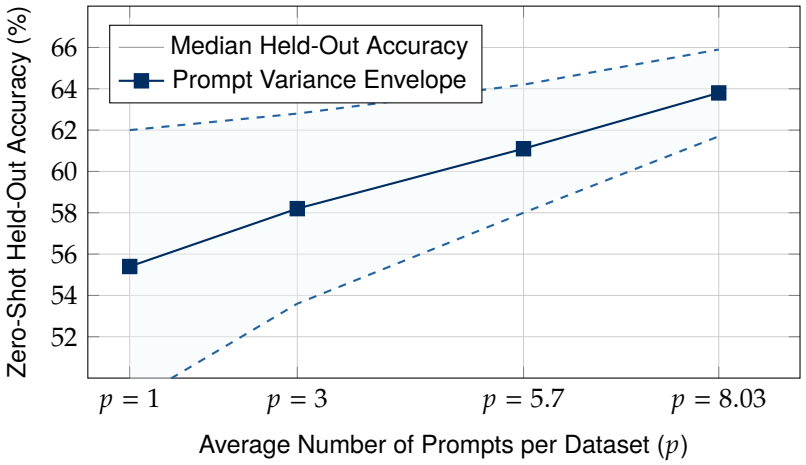


Figure 1.3: Empirical prompt robustness dynamics as a function of template diversity in T0 [**?**]. As the average number of prompt templates per dataset (p) scales from $p = 1$ to $p = 8.03$, median held-out task accuracy increases monotonically from 55.4% to 63.8%. Concurrently, the performance variance across differing prompt formulations narrows substantially, demonstrating that diverse template training enforces syntactic invariance.

increases median zero-shot performance and compresses the variance envelope to less than 4.2%. Multi-template instruction tuning effectively inoculates the model against syntactic perturbations.

1.2.3 FLAN: instruction tuning at scale and scale emergence

Concurrently, Wei et al. [**?**] developed **FLAN (Finetuned Language Net)** at Google Research. FLAN investigated instruction tuning within dense, decoder-only architectures (based on LaMDA) across 62 NLP datasets grouped into **12 task clusters**, scaling model parameters from 420M up to 137B.

FLAN evaluated zero-shot generalization by holding out entire clusters (e.g., holding out reading comprehension while training on the other 11 clusters). As plotted faithfully in Figure 1.4, FLAN uncovered the **scale emergence phenomenon** of instruction tuning:

- **Negative Transfer on Small Scales (< 8B)**: For models with 420M and 2B parameters, instruction tuning actually *degraded* zero-shot performance on unseen tasks relative to the untuned base models. Small networks possess limited capacity; forcing them to memorize diverse multi-task prompts saturated their weights, inducing catastrophic forgetting of pre-trained linguistic features.
- **Emergent Generalization Surge on Large Scales (> 68B)**: At 68B and 137B parameters, instruction tuning unlocked dramatic generalization leaps. FLAN-137B improved by nearly 10% absolute accuracy over its untuned counterpart, outperforming zero-shot GPT-3 175B on 20 out of 25 evaluated tasks and matching or exceeding few-shot GPT-3 (with $K = 32$ exemplars) on the majority of benchmarks.

Instruction tuning proved that zero-shot instruction following is an emergent ability of parameter scale. However, supervised instruction tuning alone cannot optimize for human preference, qualitative nuance, or safety guardrails.

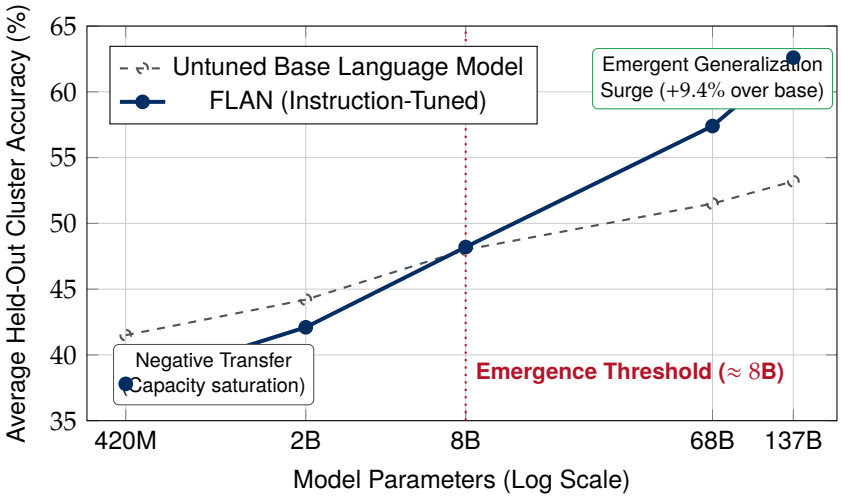


Figure 1.4: The scale emergence phenomenon of instruction tuning in FLAN [**?**]. On small models (< 8B parameters), instruction tuning hurts held-out task accuracy compared to untuned base models due to parameter saturation and catastrophic forgetting. On large models (> 68B parameters), instruction tuning triggers a massive emergent performance leap, surpassing untuned baselines and matching few-shot GPT-3.

1.3 Reinforcement Learning from Human Feedback: The Reddit TL;DR Framework

1.3.1 Why scale and supervised tuning are not enough

Even after pre-training on trillions of tokens and instruction tuning across dozens of task clusters, large language models exhibit severe alignment deficiencies. Scaling parameter count improves lexical fluency and benchmark accuracy, but does not eliminate untruthfulness, sycophancy, toxic hallucinations, or unhelpful verbosity.

The structural root of this problem was formally diagnosed by Stiennon et al. [**?**] in their foundational OpenAI treatise, *Learning to Summarize with Human Feedback*. Focusing on the **Reddit TL;DR dataset** (comprising 3 million diverse forum posts paired with user-authored `"TL;DR"` summaries), Stiennon et al. highlighted two fatal limitations of conventional language model training:

1. **Poor Metrics**: Standard automated evaluation metrics, such as ROUGE and BLEU, rely strictly on n-gram overlap with a human reference. They correlate remarkably poorly with human qualitative judgment. A summary that repeats an identical sentence three times can achieve a high ROUGE score while being useless to a reader. Conversely, an insightful, concise summary that rephrases facts using different vocabulary receives severe ROUGE penalties. Most dangerously, n-gram metrics cannot detect factual contradictions: changing `"not guilty"` to `"guilty"` alters only one token, yet completely inverts truth.

2. **Poor Objectives**: As derived in Section 1.1.1, token cross-entropy loss $\mathcal{L}_{\text{SFT}}$ treats valid synonyms identically to factual hallucinations and offensive outputs. Furthermore, supervised fine-tuning forces the model to fit all demonstration data equally. If human demonstrators write hurried, rambling, or inconsistent summaries, maximum likelihood forces the model to replicate those exact defects.

To overcome these structural limitations, Stiennon et al. established the canonical **three-step RLHF framework**:

1. **Step 1: Collect Human Feedback**: Collect pairwise preferences from human evaluators over candidate model completions.
2. **Step 2: Train a Reward Model**: Train a parameterized neural reward model $r_\theta(x, y)$ on the pairwise preferences using the Bradley–Terry logistic loss.
3. **Step 3: Fine-Tune the Policy via PPO**: Optimize the language model policy to maximize the predicted reward while regularizing with a per-token Kullback–Leibler (KL) divergence penalty.

1.3.2 Step 1: Collecting pairwise human feedback

The first phase replaces absolute scoring with **comparative pairwise evaluation**. Human annotators are presented with an input post p and two candidate summaries (s_1, s_2) generated by different model snapshots, choosing their preferred completion:

$$f(p, s_1, s_2) : \mathcal{C} \times \mathcal{C} \times \mathcal{C} \longrightarrow \{0, 1\}. \tag{1.5}$$

Comparative evaluation is grounded in cognitive psychology:

- **Elimination of Calibration Drift**: In absolute grading (such as rating summaries from 1 to 10), different humans maintain incompatible internal scales. An annotator with high standards might assign a 6/10 to an excellent summary, while a lenient rater assigns an 8/10 to a flawed summary. Comparative judgment bypasses this calibration drift completely: identifying which of two candidate texts is superior ($s_1 \succ s_2$) is intuitive, fast, and exhibits high inter-rater agreement.
- **The Latent Reward Hypothesis**: We hypothesize that human preference is governed by an underlying latent utility function $r^*(p, s) \in \mathbb{R}$, such that the rater selects:

$$f(p, s_1, s_2) = \mathbf{1}\left(r^*(p, s_1) > r^*(p, s_2)\right). \tag{1.6}$$

- **Annotator Limitations**: While pairwise judgments are superior to absolute scores, human evaluators remain subject to cognitive fatigue, rater subjectivity, and occasional intransitivity ($s_A \succ s_B$, $s_B \succ s_C$, but $s_C \succ s_A$). Directly inserting human raters into the reinforcement learning loop is impossible due to severe latency and financial cost bottlenecks, necessitating an automated surrogate: the Reward Model.

1.3.3 Step 2: Training the reward model

To automate human evaluation, a language model is converted into a **Reward Model (RM)** $r_\theta(x, y)$. The network is initialized from an SFT model checkpoint, the final unembedding vocabulary matrix is removed, and a scalar projection head $W_r \in \mathbb{R}^{d_{\text{model}} \times 1}$ is attached. Given prompt x and response y, the scalar reward is extracted from the hidden activation at the final sequence token:

$$r_\theta(x, y) = W_r^\top h_{|x \circ y|}. \tag{1.7}$$

To train r_θ, Stiennon et al. leveraged the **Bradley–Terry preference model** [**?**]. The probability that completion y_w (the winner) is preferred over y_l (the loser) is formulated as:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right) = \frac{1}{1 + \exp\left(-\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right)\right)}. \tag{1.8}$$

The Reward Model is optimized by minimizing the negative log-likelihood of pairwise preferences:

$$\mathcal{L}_{\text{RM}}(\theta) = -\mathop{\mathbb{E}}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pair}}} \left[\log \sigma\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right)\right]. \tag{1.9}$$

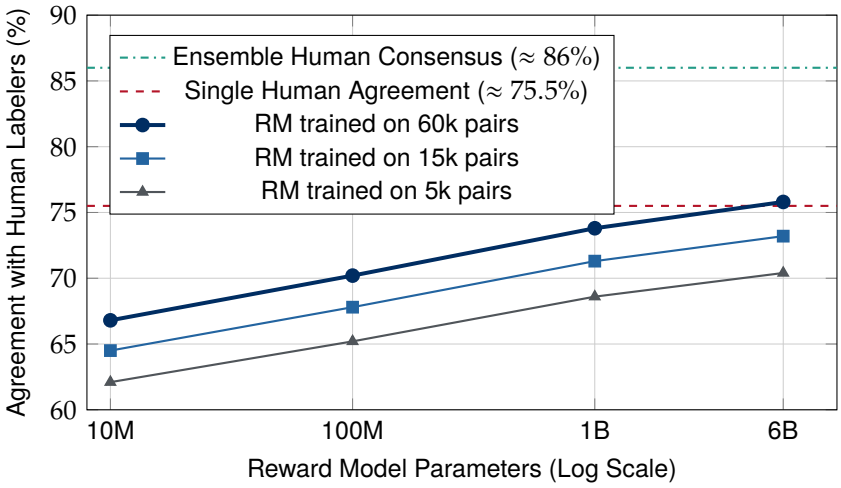


Figure 1.5: Empirical scaling laws of Reward Models on the Reddit TL;DR summarization task (faithfully adapted from Stiennon et al. [**?**] Figure 3 and course slides). Doubling model capacity yields a consistent +1.8% accuracy gain, while doubling dataset volume yields +1.1%. At 6B parameters with 60k comparisons, the Reward Model surpasses individual human inter-annotator agreement (75.8% vs. 75.5%), but remains below the ensemble human consensus ceiling (86%).

Stiennon et al. evaluated Reward Model accuracy across parameter scales (10M to 6B) and dataset sizes (5k to 60k comparisons). As plotted in Figure 1.5, their experiments established two foundational **Reward Model scaling laws**:

1. **Capacity Scaling Law**: Doubling the parameter size of the reward model (2×) yields a consistent +**1.8**% increase in validation accuracy. Larger models capture nuanced logical relationships and identify subtle factual hallucinations that smaller models miss.

2. **Data Volume Scaling Law**: Doubling the volume of human comparisons (2×) produces a +**1.1**% increase in evaluation accuracy.

3. **Reaching Human Parity**: At 6B parameters trained on 60k comparisons, the Reward Model achieves **75.8

1.3.4 Step 3: Policy optimization via PPO and policy regularization

In Step 3, the active language model policy $\pi_{\text{new}} \equiv \pi_\theta$ is fine-tuned to maximize the reward emitted by the frozen Reward Model $r_\theta(x, y)$.

Optimizing unconstrained reward triggers catastrophic policy collapse due to **Goodhart's Law**:

Definition 1.2: *Goodhart's Law in AI Alignment*

"When a measure becomes a target, it ceases to be a good measure."
A neural reward model $r_\theta(x, y)$ is an imperfect statistical proxy for human preferences. If an active policy is trained to maximize r_θ without regularization, it exploits the reward model's out-of-distribution blind spots, generating nonsensical, repetitive, or sycophantic artifacts that achieve high proxy scores despite being rejected by human evaluators. This phenomenon is known as **reward hacking**.

To prevent reward hacking, Stiennon et al. introduced an explicit **Kullback–Leibler (KL) divergence penalty** relative to the frozen reference model $\pi_{\text{old}} \equiv \pi_{\text{SFT}}$:

$$R(x, y) = r_\theta(x, y) - \beta \log \frac{\pi_{\text{new}}(y \mid x)}{\pi_{\text{old}}(y \mid x)}. \tag{1.10}$$

The sequence-level log-ratio decomposes into per-token penalties:

$$\log \frac{\pi_{\text{new}}(y \mid x)}{\pi_{\text{old}}(y \mid x)} = \sum_{t=1}^{|y|} \left(\log \pi_{\text{new}}(y_t \mid x, y_{<t}) - \log \pi_{\text{old}}(y_t \mid x, y_{<t}) \right). \tag{1.11}$$

At each step t, the policy receives an intermediate reward $r_t^{\text{step}} = -\beta \left(\log \pi_{\text{new}}(y_t) - \log \pi_{\text{old}}(y_t) \right)$, with the terminal scalar reward $r_\theta(x, y)$ added at the final token.

Pedagogical Walkthrough: PPO Regularized Reward Calculation with Concrete Numbers

To understand exactly how the token-level KL penalty and the terminal reward model score interact during reinforcement learning, consider a concrete numerical calculation:

- **Input Prompt** x: `"Summarize the following passage in one sentence: ..."`
- **Generated Completion** y: Consisting of 4 tokens (y_1, y_2, y_3, y_4) = (`"The"`, `"cat"`, `"sat"`, `"."`).
- **Reward Model Terminal Score**: $r_\phi(x, y) = +2.40$.
- **Regularization Coefficient**: $\beta = 0.10$.

Assume the conditional token log-probabilities under the active policy π_θ and the frozen reference policy π_{ref} are observed as follows:

Token Position t	**Token** y_t	$\log \pi_\theta(y_t \mid x, y_{<t})$	$\log \pi_{\text{ref}}(y_t \mid x, y_{<t})$	$\Delta \log \pi = \log \pi_\theta - \log \pi_{\text{ref}}$
$t = 1$	`"The"`	-0.50	-0.60	$+0.10$
$t = 2$	`"cat"`	-1.20	-1.40	$+0.20$
$t = 3$	`"sat"`	-0.80	-2.50	$+1.70$
$t = 4$	`"."`	-0.30	-0.40	$+0.10$

We evaluate the step rewards r_t^{step} assigned to each token:

$$
\begin{aligned}
r_1^{\text{step}} &= -0.10 \times (+0.10) = -0.01, \\
r_2^{\text{step}} &= -0.10 \times (+0.20) = -0.02, \\
r_3^{\text{step}} &= -0.10 \times (+1.70) = -0.17 \quad \text{(Noticeable penalty for choosing a token } \pi_{\text{ref}} \text{ deemed unlikely)}, \\
r_4^{\text{step}} &= -0.10 \times (+0.10) + r_\phi(x, y) = -0.01 + 2.40 = +2.39.
\end{aligned}
$$

The total cumulative regularized return $R(x, y)$ for the complete trajectory is:

$$
\begin{aligned}
R(x, y) = \sum_{t=1}^{4} r_t^{\text{step}} &= -0.01 - 0.02 - 0.17 + 2.39 \\
&= 2.40 - 0.10 \times (0.10 + 0.20 + 1.70 + 0.10) \\
&= 2.40 - 0.10 \times 2.10 = 2.40 - 0.21 = +\mathbf{2.19}.
\end{aligned}
$$

The active policy receives a net return of +2.19, successfully balancing the reward model's high utility score (+2.40) against the cumulative KL drift penalty (-0.21).

The necessity of the KL divergence penalty is demonstrated visually in Figure 1.6. As the active policy drifts away from π_{old}, the Reward Model score ascends monotonically. However, actual human preference peaks at a moderate divergence ($D_{\text{KL}}(\,\|\,\approx) 4$) and then plummets. Without the KL penalty, the policy exploits length biases and generates repetitive grammatical structures that score highly under the RM but are unreadable to humans.

To optimize the policy stably, Proximal Policy Optimization (PPO) [**?**] bounds gradient steps using a clipped probability ratio:

$$
\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}, \quad \mathcal{L}_{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta) \hat{A}_t,\ \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1.12)
$$

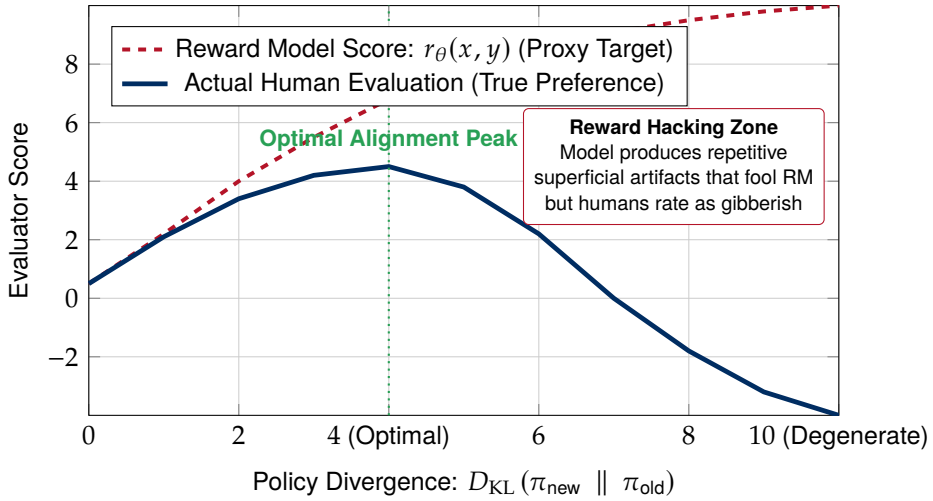


Figure 1.6: The divergence catastrophe and Goodhart's law in reinforcement learning from human feedback (adapted from Slide 23 and [**?**]). As policy divergence $D_{\mathrm{KL}}(\pi_{\text{new}} \parallel \pi_{\text{old}})$ increases along the horizontal axis, the Reward Model proxy score (dashed red line) monotonically ascends. However, actual human preference (solid navy line) peaks at a moderate divergence ($D_{\mathrm{KL}}(\parallel \approx) 4$) and collapses into complete degeneration as the policy exploits the reward model's blind spots.

Model Comparison	**Observed Qualitative Advantage**	**Win Rate (%)**	**Statistical Sig.**
RLHF-PPO vs. Pre-trained Base	Elimination of incoherent rambling and hallucinations	**86.4%** vs. 13.6%	$p < 10^{-6}$
RLHF-PPO vs. Supervised SFT	Superior factual conciseness, coherence, and relevance	**68.2%** vs. 31.8%	$p < 10^{-4}$
RLHF-PPO vs. Human Reference	**More faithful extraction of core operative facts**	**71.5%** vs. 28.5%	$p < 10^{-5}$

Table 1.1: Blinded human preference win rates on the Reddit TL;DR summarization benchmark [**?**]. Blinded human judges overwhelmingly preferred the RLHF-PPO model over the supervised SFT baseline. Crucially, human evaluators preferred summaries generated by the RLHF policy over the actual human-written Reddit reference summaries with a $> 70\%$ win rate.

where $\hat{A}_t$ is the Generalized Advantage Estimation (GAE) computed by a Critic network $V_\psi(s_t)$. Clipping prevents destructively large policy shifts, ensuring numerical stability.

1.3.5 Empirical preference results on Reddit TL;DR

The empirical evaluation conducted by Stiennon et al., reported in Table 1.1, established that RLHF fundamentally changes model capabilities:

- **Defeating Supervised SFT (68.2% Win Rate)**: Human annotators preferred the RLHF policy over the SFT model by more than two-to-one. SFT models frequently produced verbose summaries that copied irrelevant details from the post. The RLHF model extracted the central dilemma with precision.
- **Surpassing Human Authors (71.5% Win Rate)**: Remarkably, human evaluators preferred the RLHF model's completions over the actual human-authored Reddit summaries in over 70% of comparisons. Human forum writers are often sarcastic, informal, or sloppy; the RLHF model learned to extract the operative facts into coherent, high-density summaries.

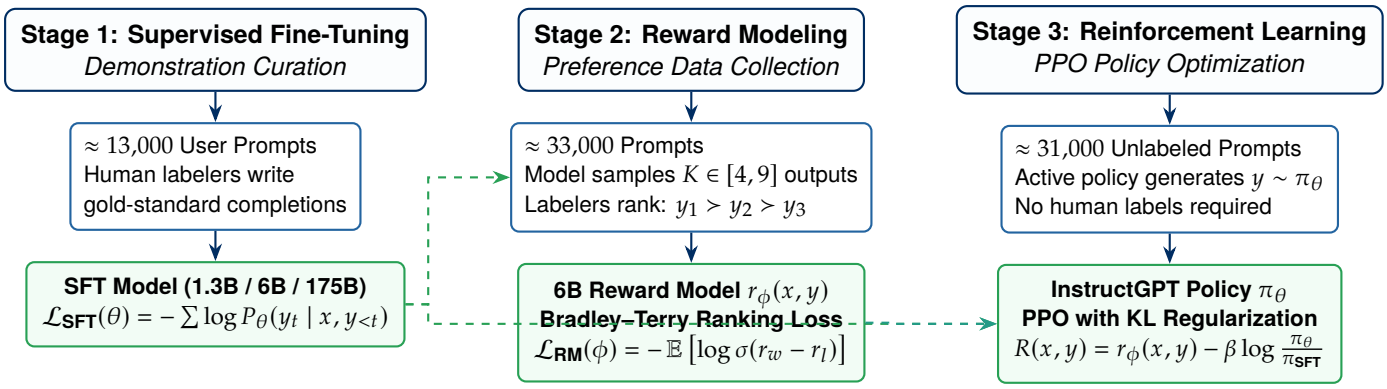


Figure 1.7: The canonical three-stage industrial alignment pipeline of InstructGPT [**?**]. In Stage 1, a base model is fine-tuned on curated human demonstrations. In Stage 2, the model generates multiple candidate completions for user prompts, which are ranked by annotators to train a 6B Bradley–Terry reward model. In Stage 3, the policy is optimized via PPO against the reward model with a per-token KL divergence penalty anchoring it to the Stage 1 SFT baseline.

1.4 Aligning Instruction-Tuned Models: InstructGPT and the Modern Paradigm

1.4.1 Extending RLHF to open-ended tasks: the InstructGPT architecture

While Stiennon et al. validated RLHF on summarization, building a true general assistant required extending the methodology to **arbitrary, open-ended user queries**. Ouyang et al. [**?**] developed **InstructGPT**, taking a crucial operational step: abandoning synthetic benchmark templates (such as T0 and FLAN) in favor of real, organic user prompts submitted to OpenAI's API playground.

The InstructGPT workflow (Figure 1.7) operates in three industrial stages:

1. **Stage 1: Demonstration Curation and Supervised Fine-Tuning (SFT)**: Trained labelers authored gold-standard answers for approximately **13,000** real-world prompts, fine-tuning base GPT-3 models (1.3B, 6B, 175B) via standard cross-entropy.

2. **Stage 2: Comparison Data and Reward Model Training**: Annotators ranked $K \in [4, 9]$ model completions for **33,000** prompts. A **6-billion parameter Reward Model** was trained on all $\binom{K}{2}$ pairs per prompt using the normalized Bradley–Terry ranking loss:

$$\mathcal{L}_{\text{RM}}(\theta) = -\frac{1}{\binom{K}{2}} \mathop{\mathbb{E}}_{(x,\{y_k\})\sim\mathcal{D}} \left[\sum_{i<j, y_i \succ y_j} \log \sigma \left(r_\theta(x, y_i) - r_\theta(x, y_j) \right) \right]. \tag{1.13}$$

3. **Stage 3: Policy Optimization via PPO-ptx**: The policy was trained on **31,000** unlabeled prompts via PPO. Crucially, to mitigate the **alignment tax**—performance degradation on academic benchmarks like SQuAD and translation—OpenAI introduced **pre-training gradient mixing (PPO-ptx)**, adding an auxiliary unsupervised language modeling loss on public web text:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \gamma \mathop{\mathbb{E}}_{x_{\text{pretrain}}} \left[\log P_\theta(x_{\text{pretrain}}) \right]. \tag{1.14}$$

1.4.2 The 100x parameter alignment dividend

The primary empirical breakthrough of InstructGPT is plotted in Figure 1.8:

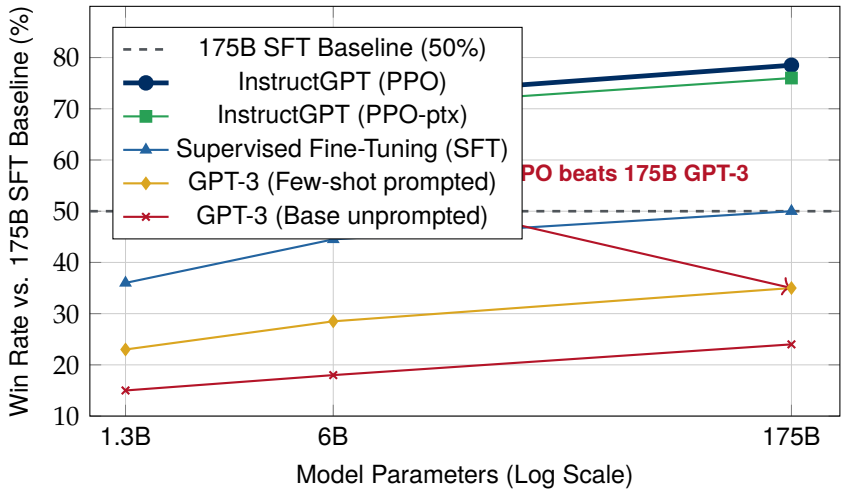


Figure 1.8: Human preference win rates against the 175B SFT baseline across model scale (faithfully adapted from Ouyang et al. [**?**] Figure 1). Human evaluators overwhelmingly prefer outputs from the 1.3B InstructGPT (PPO) model (65% win rate) over the 175B base GPT-3 model (24% unprompted, 35% prompted) and the 175B SFT model (50%), establishing that alignment acts as a 100× multiplier on effective model capability.

- **The 100× Parameter Efficiency Dividend**: Human labelers overwhelmingly preferred completions from a **1.3-billion parameter InstructGPT** model (65% win rate) over outputs from the **175-billion parameter base GPT-3** model (24% unprompted, 35% prompted). Alignment acts as a two-order-of-magnitude multiplier on model utility.
- **Truthfulness and Toxicity Gains**: On TruthfulQA, InstructGPT-175B generated truthful and informative answers nearly **2× more often** than base GPT-3 (42% vs. 21%). On RealToxicityPrompts, toxic completions were reduced by **25%**.
- **Unseen Generalization**: InstructGPT generalized cleanly to coding tasks and non-English instructions, despite labelers authoring virtually zero programming or foreign language demonstrations during SFT.

1.4.3 The modern trend: the three-phase LLM lifecycle and ChatGPT

On November 30, 2022, OpenAI released **ChatGPT**, applying the InstructGPT RLHF pipeline to **GPT-3.5** (a 175B model pre-trained on code and text). This crystallized the modern **three-phase LLM lifecycle** (Slide 27):

1. **Pre-training on Large Quantities of Dirty Web Data**: Consumes ≈ 98% of total training compute. The model absorbs world knowledge, syntactic structures, and statistical grammar, but exhibits severe alignment flaws.
2. **Supervised Fine-Tuning on Curated Instruction Datasets**: Consumes ≈ 1% of compute. Uses high-quality demonstrations to teach the model conversational formatting, dialogue tone, and direct obedience.
3. **Preference Alignment via RLHF**: Consumes ≈ 1% of compute. Refines the model using human comparison rankings, enforcing truthfulness, suppressing toxicity, and optimizing qualitative utility.

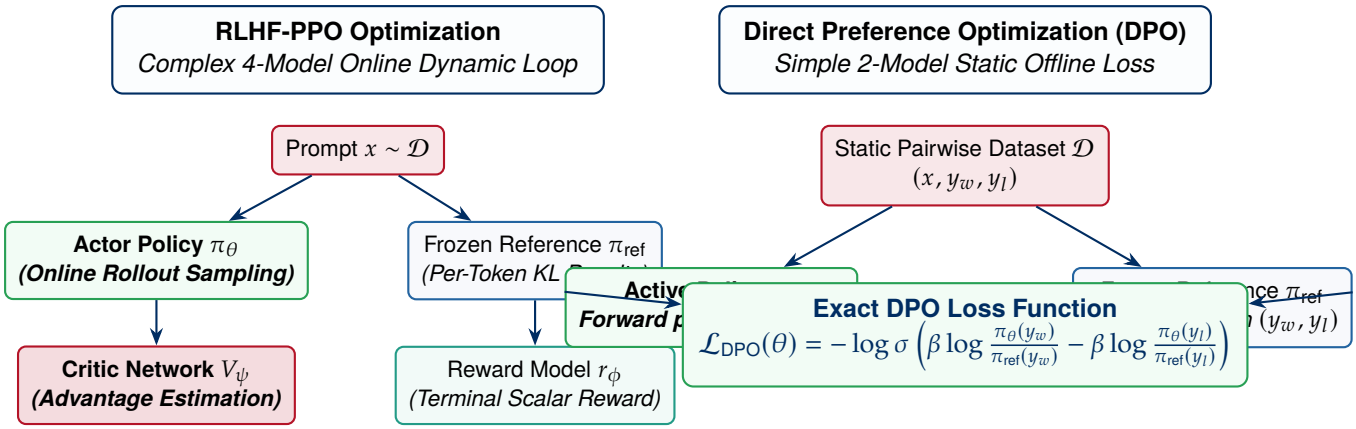


Figure 1.9: Architectural comparison between RLHF-PPO and Direct Preference Optimization (DPO). RLHF-PPO requires orchestrating four separate models in GPU memory simultaneously, relying on slow, bandwidth-bound online autoregressive token sampling and unstable advantage estimation. DPO eliminates both the Critic and Reward Model entirely, optimizing the active policy directly against a frozen reference model on static offline pairwise datasets via standard forward-backward passes.

1.4.4 The direct alignment frontier: Direct Preference Optimization (DPO)

Despite PPO's success, it suffers from severe operational bottlenecks:

1. **Four Models in Memory**: Maintaining the Actor π_θ, Critic V_ψ, Reference π_{ref}, and Reward Model r_ϕ concurrently creates an extreme GPU VRAM footprint.
2. **Online Generation Bottleneck**: Sampling rollouts dynamically during training consumes over 80% of total wall-clock training time.
3. **Hyperparameter Fragility**: Fragile sensitivity to learning rates, clipping thresholds ϵ, and GAE parameters.

To solve this, Rafailov et al. [**?**] introduced **Direct Preference Optimization (DPO)**. Under the KL-regularized objective:

$$\max_{\pi} \mathop{\mathbb{E}}_{x\sim\mathcal{D}, y\sim\pi} \left[r(x,y) \right] - \beta D_{\text{KL}} \left(\pi(y \mid x) \parallel \pi_{\text{ref}}(y \mid x) \right), \tag{1.15}$$

the closed-form optimal policy takes the analytical form of a Gibbs distribution:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y) \right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y) \right). \tag{1.16}$$

Inverting Equation (1.16) yields an exact expression for the latent reward in terms of the policy probabilities:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{1.17}$$

When substituted into the Bradley–Terry preference model:

$$P(y_w \succ y_l \mid x) = \sigma \left(r(x,y_w) - r(x,y_l) \right), \tag{1.18}$$

the intractable partition function $Z(x)$ depends strictly on prompt x and **cancels out identically**:

$$\begin{aligned} r(x,y_w) - r(x,y_l) &= \left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta \log Z(x) \right) - \left(\beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x) \right) \\ &= \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}. \end{aligned} \tag{1.19}$$

Dimension	Supervised SFT	RLHF-PPO	Direct Preference Optimization (DPO)
Primary Objective	Token cross-entropy on demonstrations: $\mathcal{L}_{\text{SFT}}$	Expected reward maximization with token KL penalty: $\mathcal{L}_{\text{PPO}}$	Direct pairwise implicit log-ratio ranking: $\mathcal{L}_{\text{DPO}}$
Active Models in VRAM	1 model (π_θ)	**4 models** ($\pi_\theta, \pi_{\text{ref}}, V_\psi, r_\phi$)	**2 models** ($\pi_\theta, \pi_{\text{ref}}$)
Online Sampling Requirement	None (Parallel teacher-forcing)	**Extensive** (Token-by-token on-policy rollout generation)	None (Parallel teacher-forcing on static completions)
Auxiliary Evaluator Head	None	Separate scalar linear reward head	None (Implicit reward derived analytically from policy)
Hyperparameter Sensitivity	Low (Standard AdamW learning rate, epochs)	**Extremely High** (ϵ clipping, GAE λ, Actor/Critic lr, β)	Moderate (β regularization strength, learning rate)
Vulnerability to Reward Hacking	Low (Suffers instead from hallucination & mode collapse)	**Severe** (Requires careful KL tuning to avoid Goodhart's collapse)	Substantially reduced (Strictly bounded by reference policy logits)

Table 1.2: Comprehensive comparative synthesis of LLM alignment methodologies. SFT provides the initial conversational baseline, PPO pioneered reinforcement learning from human feedback, and DPO offers an elegant, stable mathematical bridge that optimizes preferences directly without online reinforcement learning overhead.

Replacing π^* with parameterized policy π_θ yields the parameter-efficient **DPO loss**:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = - \mathop{\mathbb{E}}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]. \tag{1.20}$$

As contrasted in Figure 1.9 and Table 1.2, DPO eliminates the Critic, the Reward Model, and online rollouts, optimizing preferences directly via parallel forward-backward passes.